ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 15
03/05/2026

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Old policy, we require: s_t, a_t and s_{t+1}

With s_t, a_t and s_{t+1} we can calculate the reward $r_t = r(s_t, a_t, s_{t+1})$

Q Learning

Discount factor, $\gamma \in [0,1]$

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Learning rate, $\alpha \in [0,1]$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot \underbrace{Q^{old}(s_t, a_t)}_{\text{Previous Estimate of Value of Action } a_t \text{ when in state } s_t} + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot \underbrace{[r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]}$$

Updated Estimate of Value of
Action a_t when in state s_t
After Observing r_t

Q Learning

Learning rate, $\alpha \in [0,1]$

Discount factor, $\gamma \in [0,1]$

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot \underbrace{Q^{old}(s_t, a_t)}_{\text{Previous Estimate of Value of Action } a_t \text{ when in state } s_t} + \alpha \cdot \underbrace{[r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]}_{\text{Updated Estimate of Value of Action } a_t \text{ when in state } s_t \text{ After Observing } r_t}$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- We do not need to perform the action defined by $\operatorname{argmax}_a Q^{old}(s_{t+1}, a)$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- We do not need to perform the action defined by $\underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$
- Next action a_{t+1} may be based on $\underset{a_{t+1}}{\operatorname{argmax}}\, Q^{new}(s_{t+1}, a_{t+1})$, we then observe immediate reward r_{t+1} and new state s_{t+2}

$$\ldots\ s_t \quad a_t \quad r_t \quad s_{t+1} \quad a_{t+1} \quad r_{t+1} \quad s_{t+2}\ \ldots$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- We do not need to perform the action defined by $\underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$
- Next action a_{t+1} may be based on $\underset{a_{t+1}}{\operatorname{argmax}}\, Q^{new}(s_{t+1}, a_{t+1})$, we then observe immediate reward r_{t+1} and new state s_{t+2}

$$\ldots\ s_t \quad a_t \quad r_t \quad s_{t+1} \quad a_{t+1} \quad r_{t+1} \quad s_{t+2}\ \ldots$$

- Sequentially and continually update the Q function $Q^{new}(s_t, a_t)$, which is an $n \times m$ matrix

Learning Rate

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Learning rate, $\alpha \in [0,1]$

- Convergence guaranteed after sufficient experience, if learning rate diminishes with time
- In practice one may set $\alpha = 0.1$
- This online Q-learning setup allows one to learn an optimal policy based directly on experience

Further Insights on Q-Learning

- Recall that Q learning is characterized by the update equation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Further Insights on Q-Learning

- Recall that Q learning is characterized by the update equation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- For simplicity, consider the case $\alpha = 1$ and hence the update

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

Further Insights on Q-Learning

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

Further Insights on Q-Learning

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

- For all actions $a \in A$ we similarly have

$$Q^{old}(s_{t+1}, a_{t+1}) \leftarrow r(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma \cdot \max_a Q^{older}(s_{t+2}, a)$$

Repeated Use of Update Equation

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a \boxed{Q^{old}(s_{t+1}, a)}$$

- For all actions $a \in A$ we similarly have

$$\boxed{Q^{old}(s_{t+1}, a_{t+1})} \leftarrow r(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma \cdot \max_a Q^{older}(s_{t+2}, a)$$

Looking Two Steps Ahead

- Using the expression for $Q^{old}\ (s_{t+1}, a)$, we therefore have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a \underbrace{[\,\gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]}_{= \gamma \cdot Q^{old}\ (s_{t+1}, a)}$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a \left[\gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a') \right]$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \cdot r(s_{t+2}, a', s_{t+3}) + \ldots$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \; r(s_{t+2}, a', s_{t+3}) + \ldots$$

Reward scales by multiplicative factor γ^t for t steps into the future

Result of Sequence of Iterations

- We therefore have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \cdot r(s_{t+2}, a', s_{t+3}) + \ldots$$

Implement the max operations from horizon to the present time (right to left)
Termed a "backup algorithm"

Meaning of Q Function

- With sufficient training data (experience), the Q function represents

$$Q(s,a) = Average\left[r(s,a,s_{t=1}) + \underbrace{\sum_{t=1}^{\infty} \gamma^t r(s_t,a_t,s_{t+1})}_{\text{Actions based on optimal policy } \pi(a;s)}\right]$$

- Average taken over environment statistics $p(s,a,s')$, which are not known explicitly, but experienced when learning

Meaning of Q Function

- With sufficient training data (experience), the Q function represents

$$Q(s,a) = Average\left[r(s,a,s_{t=1}) + \underbrace{\sum_{t=1}^{\infty} \gamma^t r(s_t,a_t,s_{t+1})}_{\text{Actions based on optimal policy } \pi(a;s)}\right]$$

- Average taken over environment statistics $p(s,a,s')$, which are not known explicitly, but experienced when learning

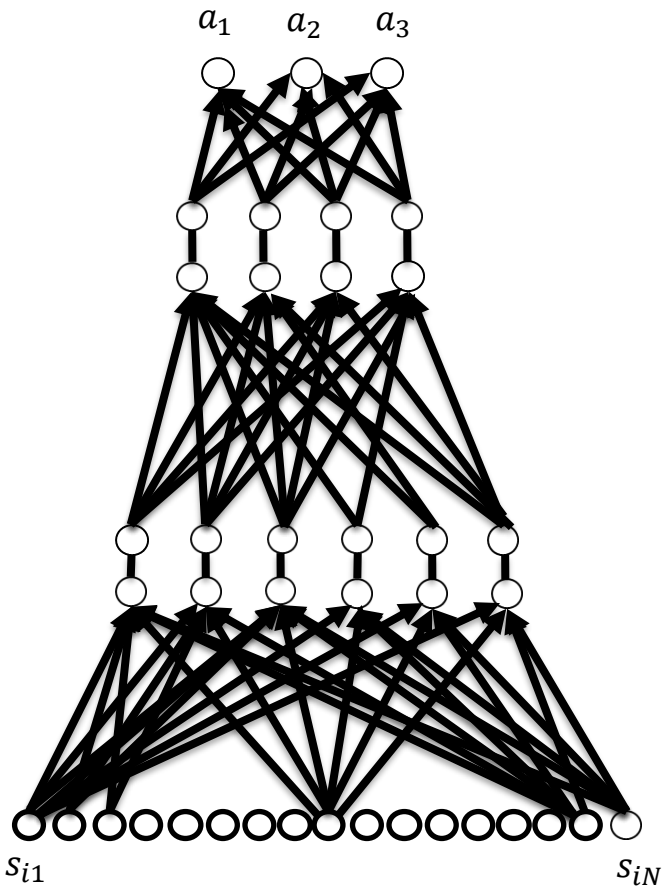
- Mathematically, the aforementioned average is termed an "expectation" wrt $p(s,a,s')$

Limitations of Tabular Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- Simple update rule, but …..
  - The Q function is stored as a table/matrix, which is impractical when considering a large number of states and actions
  - No real capacity to generalize across sequence types, because the tabular Q function does not have a functional form

Represent Q Function as a Neural Network



- The Q-function modeled via a neural network
- The state is input (*e.g.*, an image with *N* pixels), and at the output we model the Q-function $Q(s, a)$ for each possible action
- Represent the neural network model as $Q(s, a; \theta)$ where θ represent the neural network parameters we wish to learn
- After we learn θ the policy is

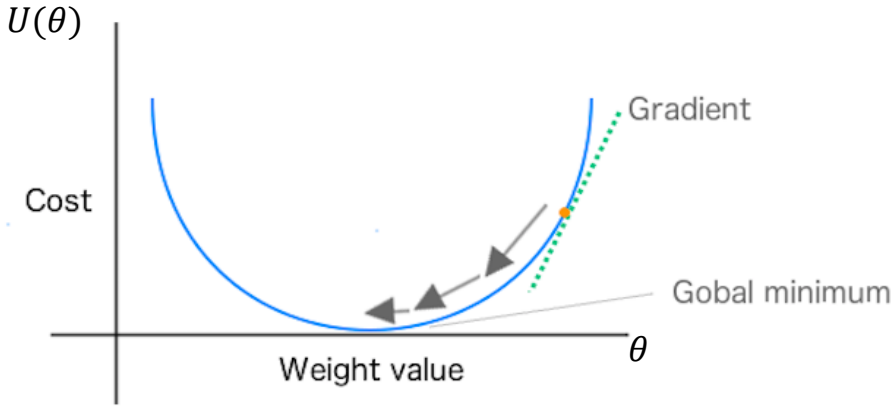
$$\pi(s) = \underset{a}{\operatorname{argmax}}\, Q(s, a; \theta)$$

Deep Q Learning (DQN)

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\,]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as



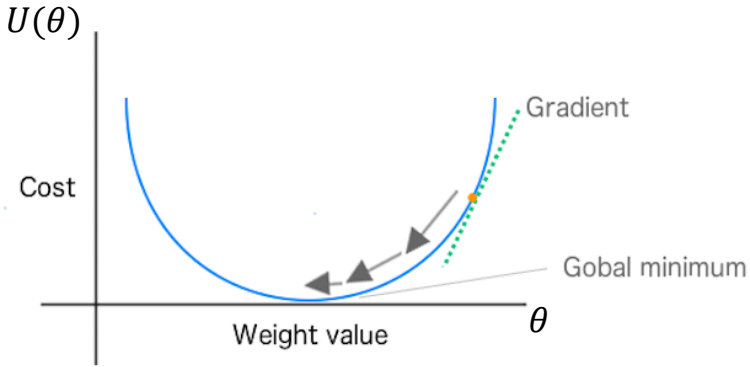
Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ \right]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \nabla_\theta U(\theta; s_t, a_t)|_{\theta=\theta^{old}}$$



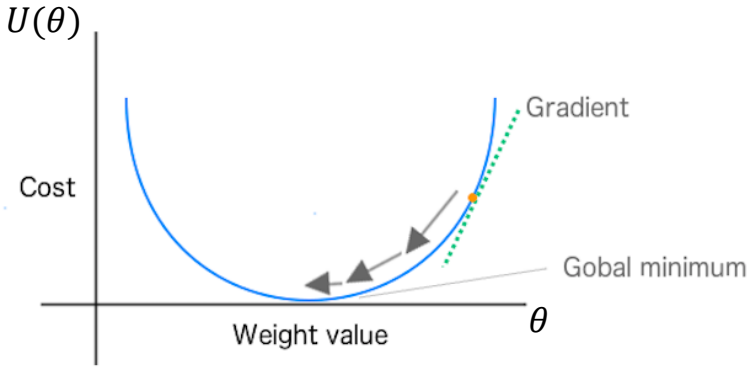
Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ \right]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \underbrace{\nabla_\theta U(\theta; s_t, a_t)|_{\theta=\theta^{old}}}_{\text{Multi-dimensional “slope”}}$$



Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ \right]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \nabla_\theta U(\theta; s_t, a_t)|_{\theta=\theta^{old}}$$

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \left[Q\left(s_t, a_t; \theta^{old}\right) - r_t - \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right)\right] \nabla_\theta \mathrm{Q}(s_t, a_t; \theta)|_{\theta=\theta^{old}}$$

Gradient Descent

$$\theta^{new} \leftarrow \theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old})\right] \nabla_\theta \mathrm{Q}(s_t, a_t; \theta)|_{\theta=\theta^{old}}$$

Looks a lot like the prior Q learning

Gradient Descent

$$\theta^{new} \leftarrow \theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old})\right] \underbrace{\nabla_\theta \mathrm{Q}(s_t, a_t; \theta)|_{\theta=\theta^{old}}}_{\text{Modification to account for neural network parameters } \theta}$$

Gradient Descent

$$\theta^{new} \leftarrow \underbrace{\theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old})\right]}_{\text{Looks a lot like the prior Q learning}} \underbrace{\nabla_\theta \mathrm{Q}(s_t, a_t; \theta)|_{\theta=\theta^{old}}}_{\text{Modification to account for neural network parameters } \theta}$$

- One may show rigorously that this is directly related to the prior sequential Q learning

- Details not very difficult (first-order Taylor series expansion), but beyond the scope of our discussion

Revisit What Deep Q Network Seeks to Learn

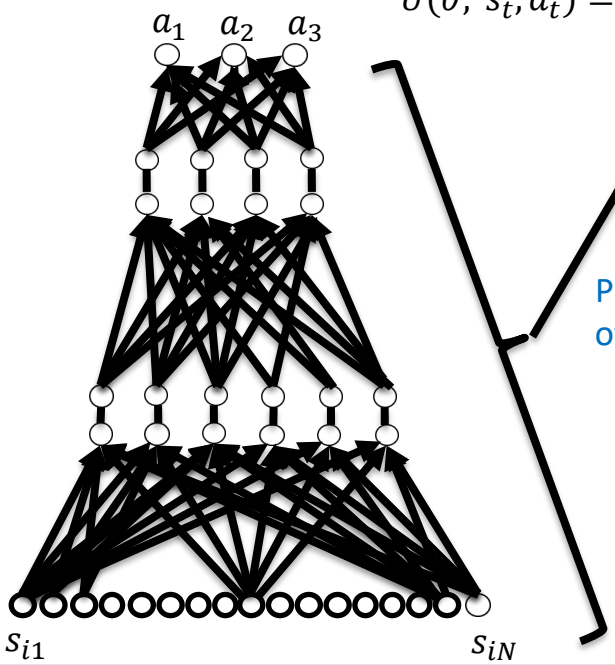
- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$

Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$



Predictive model for value of state-action pair (s_t, a_t)

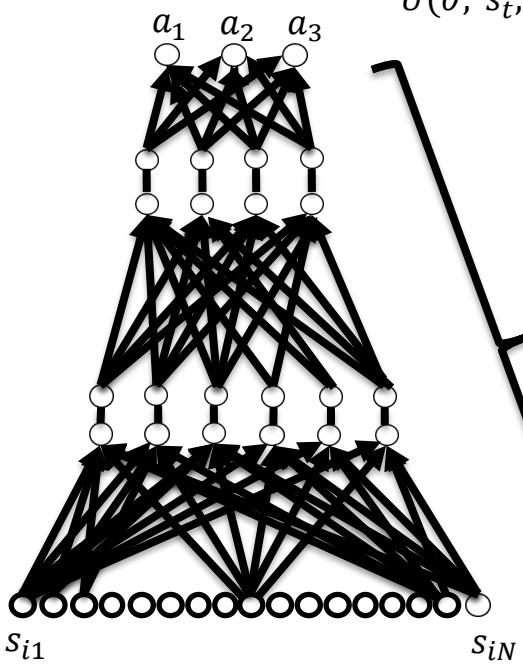
Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \overbrace{\gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})}^{\text{Discounted future rewards}}\right]\right]^2$$

Next immediate reward

Predictive model for value of state-action pair (s_t, a_t)



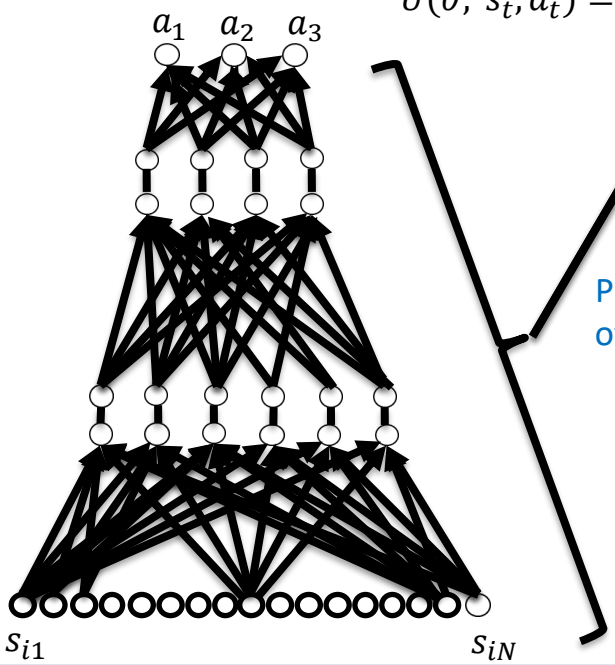
Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

Discounted future rewards

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$

Next immediate reward



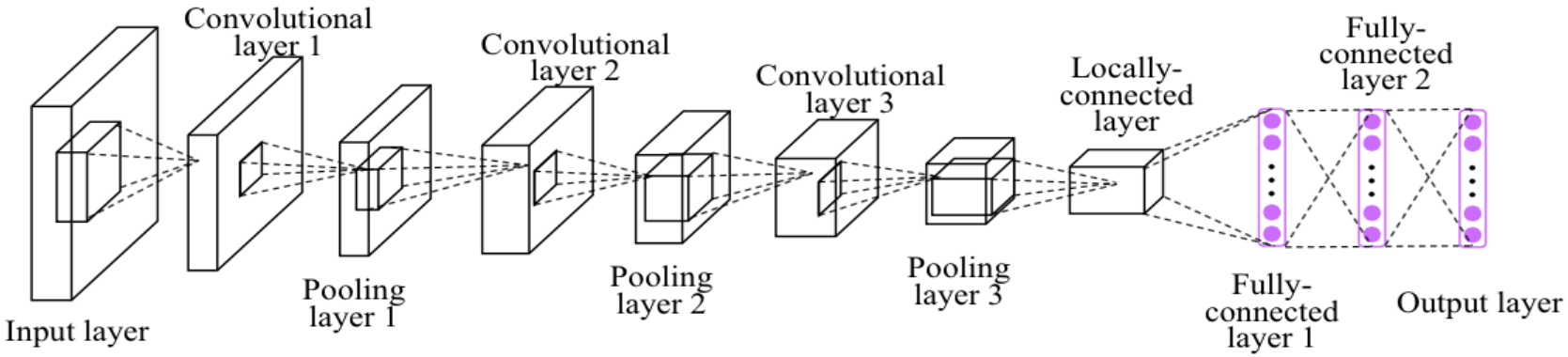
Predictive model for value of state-action pair (s_t, a_t)

Supervised Learning

Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

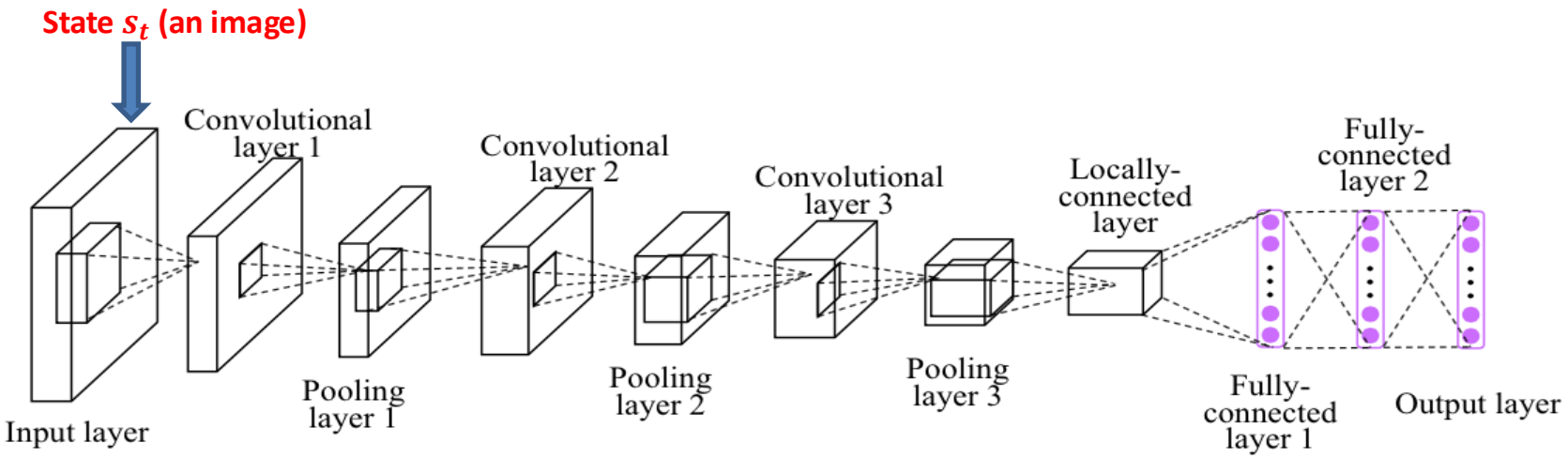
$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$



Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

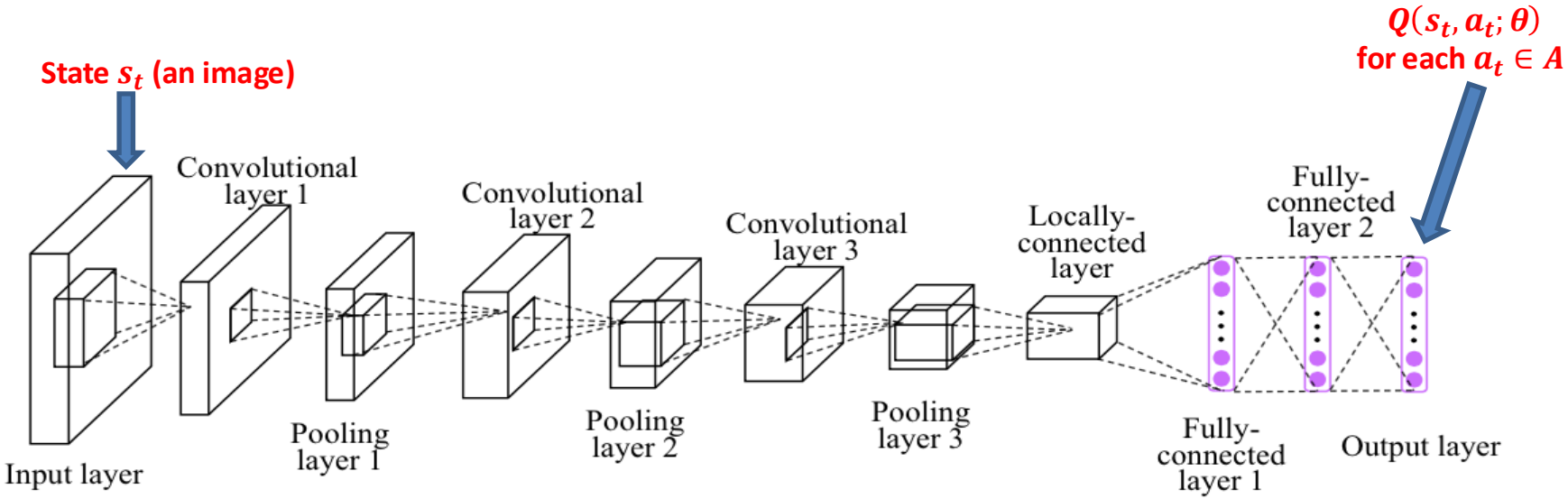
$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$



Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$



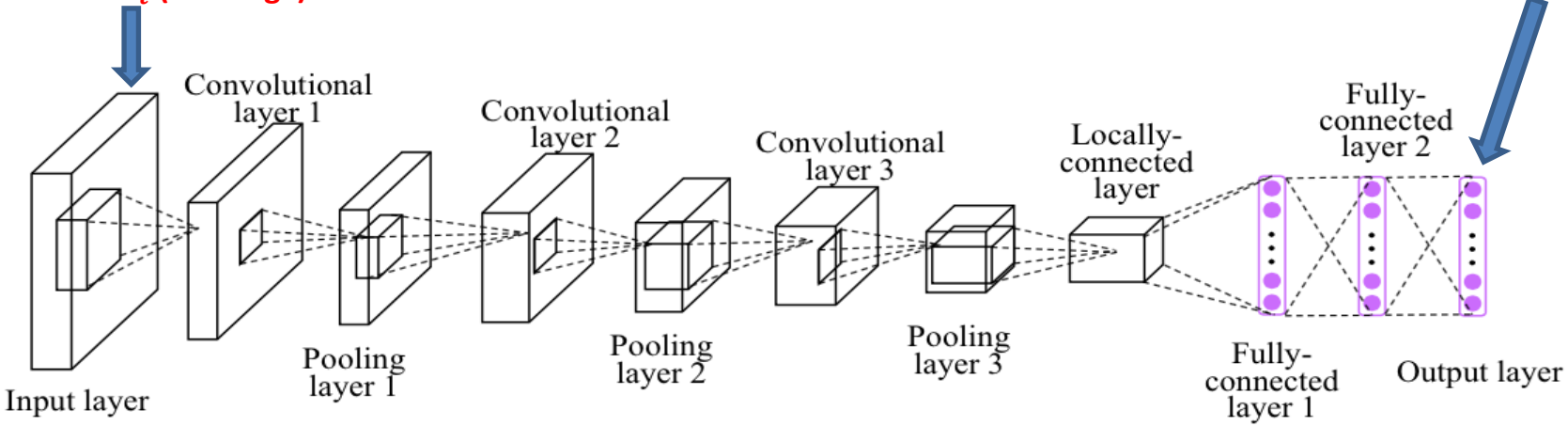
Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

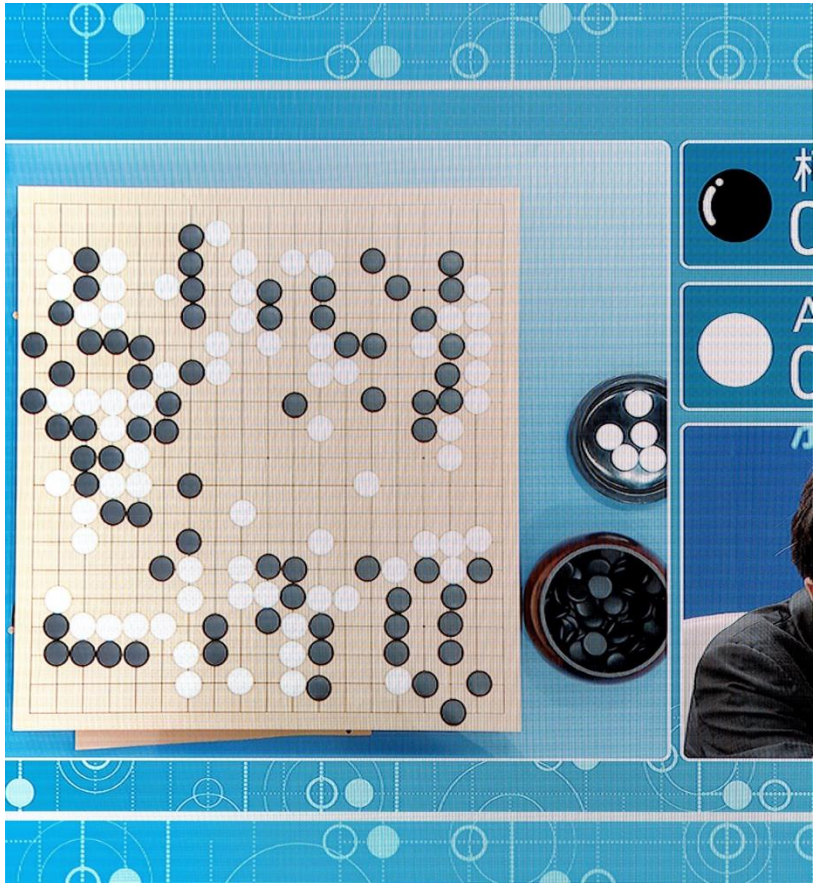
$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$

$Q(s_t, a_t; \theta)$ for each $a_t \in A$

State s_t (an image)



Google DeepMind
Challenge Match
8 - 15 March 2016
ALPHAGO
01:59:54
LEE SEDOL
01:59:02
AlphaGo



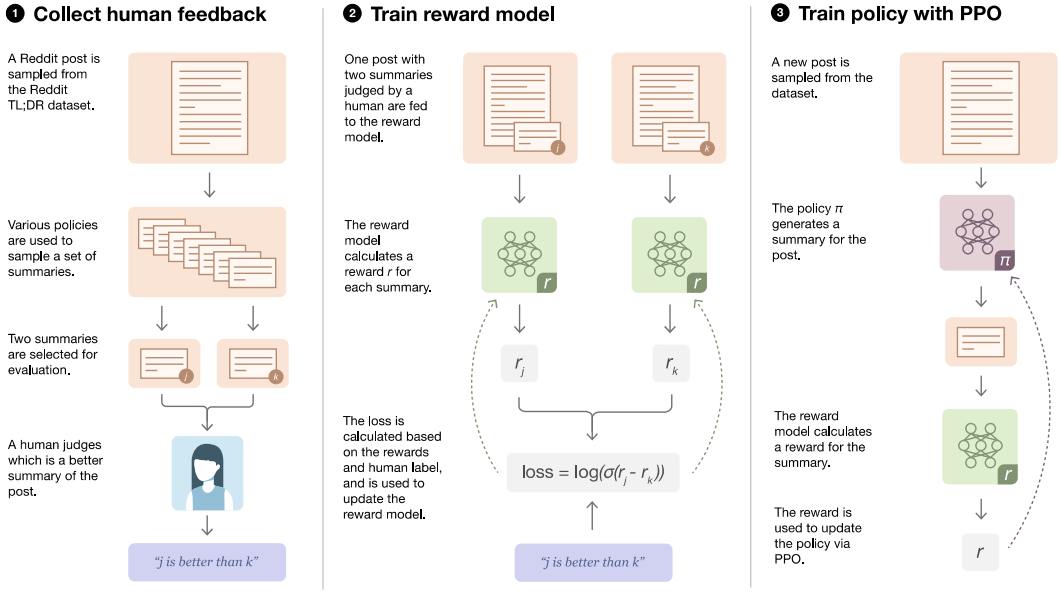
Atari Games

Extend RL for Language Modeling

- How do we extend RL ideas to the characteristics of language models?

- Would like to encourage LLM to generate text that is aligned with the desires of people

- Would like to encourage certain behaviors of the LLM: chain-of-thought reasoning



RL for Large Language Models

- With LLM, we effectively have a policy $\pi_\phi(a_t|s_t)$ where this is a deep Transformer with parameters ϕ

- The state s_t is the text input to the Transformer, and the action a_t is the <u>full</u> text response (of length n)

$$\pi_\phi(a|s) = \prod_{i=1}^{n} p(a_i|s, a_{1:i-1})$$

RL for Large Language Models

- With LLM, we effectively have a policy $\pi_\phi(a_t|s_t)$ where this is a deep Transformer with parameters ϕ

- The state s_t is the text input to the Transformer, and the action a_t is the <u>full</u> text response (of length n)

$$\pi_\phi(a|s) = \prod_{i=1}^{n} \underbrace{p(a_i|s, a_{1:i-1})}_{\text{Transformer}}$$

RL for Large Language Models

- With LLM, we effectively have a policy $\pi_\phi(a_t|s_t)$ where this is a deep Transformer with parameters ϕ

- The state s_t is the text input to the Transformer, and the action a_t is the <u>full</u> text response (of length n)

$$\pi_\phi(a|s) = \prod_{i=1}^{n} p(a_i|s, a_{1:i-1})$$

- Assume that we can manifest a reward model, $r_\beta(a, s)$, that is capable of assigning a "reward" to a generated document a for input/prompt s

- Through human-machine interactions, we may have a *sequence* of actions and rewards (chat bot), where we have series of prompts, actions (generated text), and rewards

- Wish to refine Transformer parameters ϕ to improve alignment with human wishes

Modeling the Q Function with LLM

- The Q function computes $Q(s, a)$, quality of action a for input state s

- The input state is the text prompt, and our policy (LLM) is

$$\pi_\phi(a|s) = \prod_{i=1}^{n} p(a_i|s, a_{1:i-1})$$

- One idea is to specify a model $V_\theta(s)$ which quantifies the value of repeatedly following current policy, with discounted reward

- $V_\theta(s)$ can share the same LLM as $\pi_\phi(a|s)$ to analyze s, and apply a separate linear layer at the output

- Maybe: $Q(s, a) = \pi_\phi(a|s)V_\theta(s)$

Policy and Value Function

- State-of-the-art RL for LLMs models policy $\pi_\phi(a|s)$ and value function $V_\theta(s)$

- Doesn't explicitly use the Q function, but analogously *jointly* learns $\pi_\phi(a|s)$ and $V_\theta(s)$

- More on this below, where we discuss the latest approaches in RLHF

- We first address key question: How do we assign reward (penalty) to the text generated by LLM, based on human feedback?

- Human-generated feedback is the key to designing the reward function, and to RLHF

Advantage Function

$$A_t = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1}}_{\text{Observed rewards of policy over } T-t \text{ steps}} + \gamma^{T-t} V_\theta(s_T) - V_\theta(s_t)$$

- Starting from state s_t, follow current policy (LLM) until we arrive at state s_T
- Total of $T - t$ steps in which we observe reward
- Value function $V_\theta(s)$ learned based on sequences of length much longer than T
- When refining the policy, we will consider A_t for $t \in [0, T-1]$

Advantage Function

$$A_t = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1}}_{\text{Observed rewards of policy over } T-t \text{ steps}} + \underbrace{\gamma^{T-t} V_\theta(s_T)}_{\text{Discounted Expected Value of State } s_T} - V_\theta(s_t)$$

Advantage Function

$$A_t = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)}_{\text{Empirical Value of State } s_t} \underbrace{- V_\theta(s_t)}_{\text{Prior Belief of Value of State } s_t}$$

Advantage Function

$$A_t = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V_\theta(s_T) - V_\theta(s_t)}_{\text{A generalized temporal difference}}$$

- Refine policy (generative LLM) such that sequences with positive advantage A_t are encouraged

- Experience with negative advantage is discouraged in policy refinement

Reward Model

$$A_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T) - V_\theta(s_t)$$

- A key element to the above setup concerns computing the reward (for alignment) r_t of the LLM-generated text
- Desire human feedback to assess alignment and specify reward
- Cannot have humans constantly reading LLM-generated text and providing a reward
- Need to automate reward generation for text: $r = h_\beta(a, s)$
- NN with parameters β for LLM-generated text

Human Assessment of Responses

- Consider many different prompts, $s_1, \ldots, s_M$ where each of these is to be input to the LLM to generate an output, or action

- These $s_1, \ldots, s_M$ may represent documents to be summarized, or they may be questions for which an LLM-generated answer is desired

- For each input s_m obtain a set of possible actions/outputs, $a_{1m}, \ldots, a_{Km}$

- These K responses may be generated by different LLMs and/or by humans (*e.g.*, Reddit)

Human Evaluation

- Data: Set of LLM text inputs/prompts $\{s_m\}_{m=1,M}$ and possible responses for each $\{a_{1m}, \dots, a_{Km}\}_{m=1,M}$
- For input s_m, ask human to evaluate whether a_{im} or a_{jm} is better
- Do this with many human evaluators, for all $s_1, \dots, s_M$ and all pairs a_{im} and a_{jm}
- Now develop a model which predicts which answer humans will prefer, given input state
- This model will be used within the RL setup, to provide immediate reward evaluation for particular action

Model (1/3)

- For input s_m, assume we have asked many humans to evaluate if a_{im} or a_{jm} is better
- For reward-model design, use BERT, and use [CLS] (classification) output position
- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vector for a_{im} and a_{jm}, respectively
- Learn a classifier with weight vector $w \in R^d$ applied to CLS output

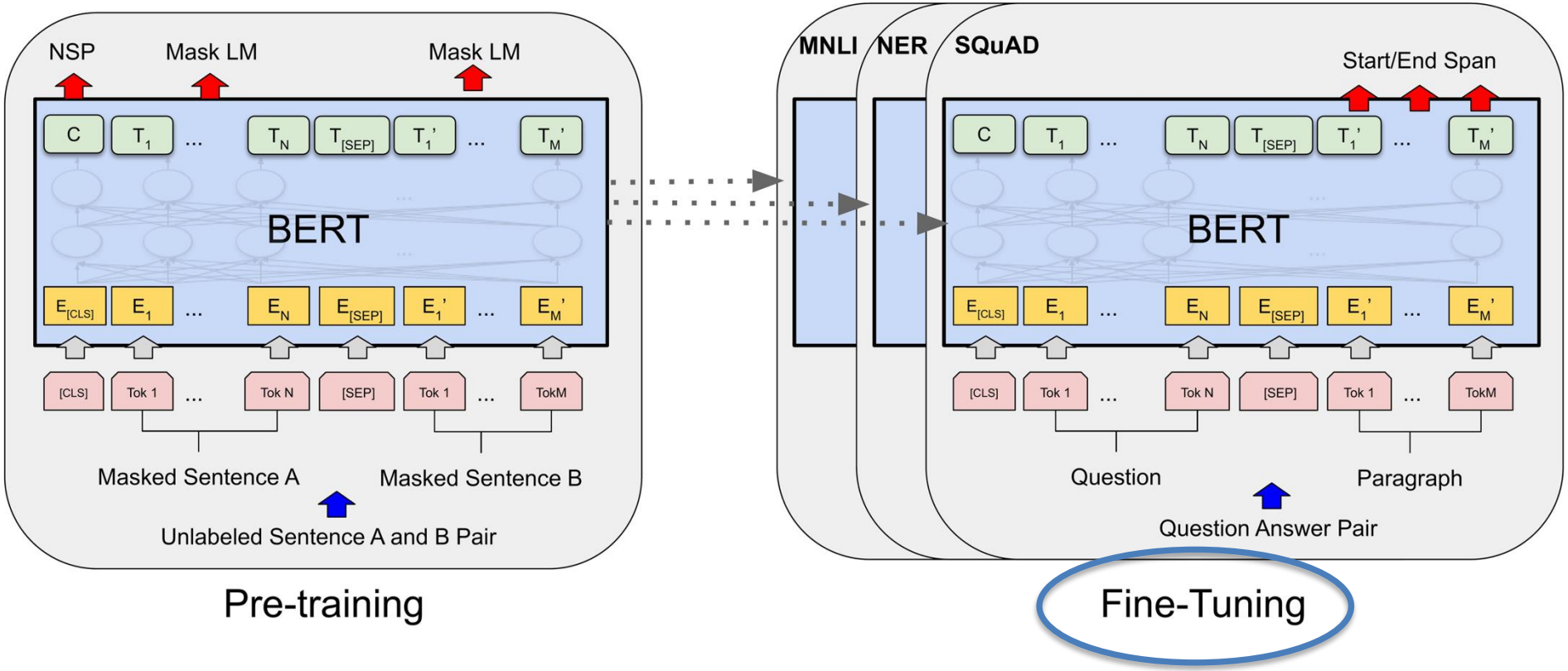
Model (2/3)

- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vectors for BERT inputs (s_m, a_{im}) and (s_m, a_{jm})

- Learn a classifier with weight vector $w \in R^d$ applied to CLS output

- Probability that a_{im} is preferred: $$\frac{\exp(w \cdot o_{im})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$$

- Probability that a_{jm} is preferred: $$\frac{\exp(w \cdot o_{jm})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$$



- Fine-tune for this human-preference test
- “Sentence A” and “Sentence B” are here the input and possible output from Transformer

Model (3/3)

- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vectors for a_{im} and a_{jm}

- Prob a_{im} is preferred: $\dfrac{\exp(w \cdot o_{im})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$ Prob a_{jm} is preferred: $\dfrac{\exp(w \cdot o_{jm})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$

- Learn w to maximize cross entropy for human-generated preferences, *i.e.*, to classify human preferences

- Reward associated with o_{im} for input s_m: $r(s_m, a_{im}) = w \cdot o_{im}$

Beyond Q Functions

- The Q function $Q_\theta(s, a)$ quantifies the value of taking action a in state s
- The LLM prediction of response may be viewed as a policy $\pi_\phi(a|s)$
- Now develop an alternative for of RL, with a better fit to aligning LLMs with human needs
- Builds naturally on what we have discussed in the context of deep Q learning

Value Function

- Introduce a value function $V_\theta(s)$
- This represents the value (expected reward) of following our current policy $\pi_\phi(a|s)$ from state s
- We separate computation of the value function and the policy (actions), rather than combining them in Q function
- Matched with LLM learning, as $\pi_\phi(a|s)$ can be computed directly via our Transformer, as can $V_\theta(s)$

Advantage Function

- Assume we have the current value function $V_\theta(s)$ and policy $\pi_\phi(a|s)$

- Iterate between updating policy and updating the value function based on new policy

- Using current policy, we take LLM actions, generate text, and get a rewards

- Calculate the advantage of updated policy (temporal difference):

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

Advantage Function

$$A_t = -\underbrace{V_\theta(s_t)} + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

Value of state (prompt) based on implementing prior policy

Advantage Function

$$A_t = -V_\theta(s_t) + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1}}_{\text{Observed discounted rewards of implementing refined policy for } T-t \text{ steps}} + \gamma^{T-t} V_\theta(s_T)$$

Advantage Function

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \underbrace{\gamma^{T-t} V_\theta(s_T)}_{\text{Discounted value of state } s_T}$$

Advantage Function

$$A_t = -\underbrace{V_\theta(s_t)}_{\text{Value of state (prompt) based on implementing prior policy}} + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)}_{\text{Empirical evaluation of current state reward}}$$

Advantage Function

$$A_t = \underbrace{-V_\theta(s_t)}_{\text{Value of state (prompt) based on implementing prior policy}} + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)}_{\text{Empirical evaluation of current state reward}}$$

- If $A_t > 0$, then the refined policy is good, and we wish to encourage the associated policy
- If $A_t < 0$, then the refined policy misaligned with human desires, wish to discourage associated policy
- The value function of state s_t is also updated using observed advantage

$$V_{\theta_{new}}(s_t) \leftarrow V_{\theta_{old}}(s_t) + A_t$$

- Refine neural network $V_\theta(s)$ representation of the value function (this is like Q learning)

Loss Function via Importance Sampling

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

$$V_{\theta_{new}}(s_t) \leftarrow V_{\theta_{old}}(s_t) + A_t$$

- Above advantage-function model and value-function model are learned using sequences generated from the previous version of the policy (here, the LLM using its prior model parameters)

- Desire to optimize LLM parameters as

Expectation with <u>new</u> policy

$$\hat{\phi} = \text{argmax}_\phi \, E_{\pi_\phi}[A]$$

- Learn LLM parameters ϕ such that associated expected advantage is high (improve expected reward relative to prior value function)

Loss Function via Importance Sampling

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

Expectation with <u>new</u> policy

$$\hat{\phi} = \text{argmax}_\phi \, E_{\pi_\phi}[A]$$

- Problem: To calculate the expectation, we need reward trajectories with the new policy, which we are trying to update

- Solution: Draw trajectories with the prior policy, and then use importance sampling to simulate desired expectation

Loss Function via Importance Sampling

$$\hat{\phi} = \text{argmax}_{\phi}\, E_{\pi_{\phi}}[A]$$

$$E_{\pi_{\phi}}[A] = E_{\pi_{\phi_{old}}}[\frac{\pi_{\phi}}{\pi_{\phi_{old}}} A]$$

$$\approx \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_{\phi}(s_n)}{\pi_{\phi_{old}}(s_n)} A(s_n)$$

The sequences used to manifest this expectation simulated using the LLM/policy based on the prior ("old") parameters π_{old}

Refine Value Function & Policy

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

$$V_{\theta_{new}}(s_t) \leftarrow V_{\theta_{old}}(s_t) + A_t$$

- In addition to updating the NN $V_\theta(s)$, wish to update the policy $\pi_\phi(a|s)$
- Assume we have computed/observed many advantage values A_t for associated states s_t
- Want to push the policy in direction of positive Advantage

$$\hat{\phi} = \text{argmax}_\phi \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_\phi(a_n|s_n)}{\pi_{\phi_{old}}(a_n|s_n)} A_n$$

Refine Value Function & Policy

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

$$\hat{\phi} = \text{argmax}_\phi \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_\phi(a_n|s_n)}{\pi_{\phi_{old}}(a_n|s_n)} A_n$$

- If advantage $A_n > 0$, then maximizing above expression means making action a_n more probable for input s_n
- If $A_n < 0$, maximizing learning function corresponds to making a_n less probable
- The scale of the advantage (positive or negative) dictates how important a given experience is to policy refinement

Clipped Policy Refinement

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

$$\hat{\phi} = \text{argmax}_\phi \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_\phi(a_n|s_n)}{\pi_{\phi_{old}}(a_n|s_n)} A_n$$

- Define the ratio of the refined and old policy as

$$r_n(\phi) = \frac{\pi_\phi(a_n|s_n)}{\pi_{\phi_{old}}(a_n|s_n)}$$

- Constrain ("clip") how much change is allowed in the policy: $1 - \epsilon \leq r_n(\phi) \leq 1 + \epsilon$

Parameter Sharing

$$A_t = -V_\theta(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\theta(s_T)$$

$$\pi_\phi(a_t|s_t) \qquad\qquad V_\theta(s_t)$$

- Iteratively assess alignment by computing advantage (discounted reward) of current policy
- Based on advantage, update NN models of value function $V_\theta(s)$ and of the policy $\pi_\phi(a|s)$
- The input to both models is the input state s, here the input text, or prompt
- <u>Share</u> most of the Transformer parameters for these two models, and employ different linear layers at output for softmax probability of next word in $\pi_\phi(a|s)$ and (separately) for the value $V_\theta(s)$ of the state